

Precious Jackson

CS 499

9/23/2026

Milestone Four: Enhancement Three – Databases

Artifact Description

The artifact selected for this enhancement is the Travlr Getaways full-stack web application that I developed during CS 465 Full Stack Development. The application uses Node.js and Express for server-side functionality, MongoDB for data storage, Mongoose for object modeling and validation, and a web interface for presenting travel information. Trip records contain information such as a trip code, name, length, start date, resort, price per person, image, and description. I selected the database portion of Travlr Getaways for this milestone because it provided a practical opportunity to strengthen data integrity and database retrieval while preserving the application's existing functionality.

For the database enhancement, I improved the Mongoose trip schema and the MongoDB trips collection so that important fields are validated and commonly accessed data can be retrieved more efficiently. The enhanced application still retrieves the stored trip records and presents them through the Travel page, which demonstrates that the database changes did not prevent the application from using its existing data.

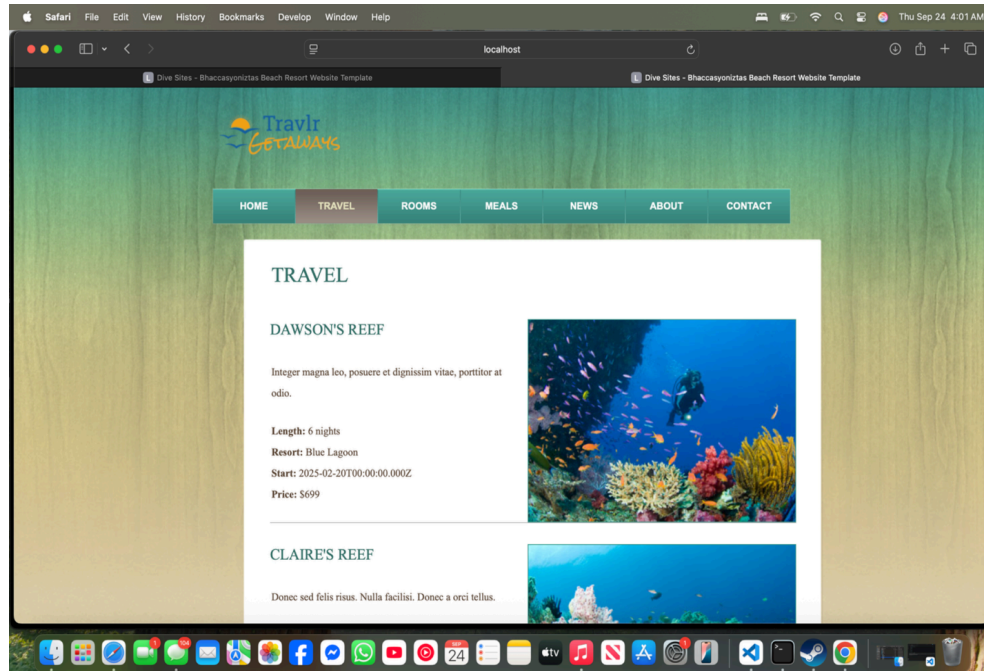


Figure 1. Travlr Getaways Travel Page After Database Enhancement.

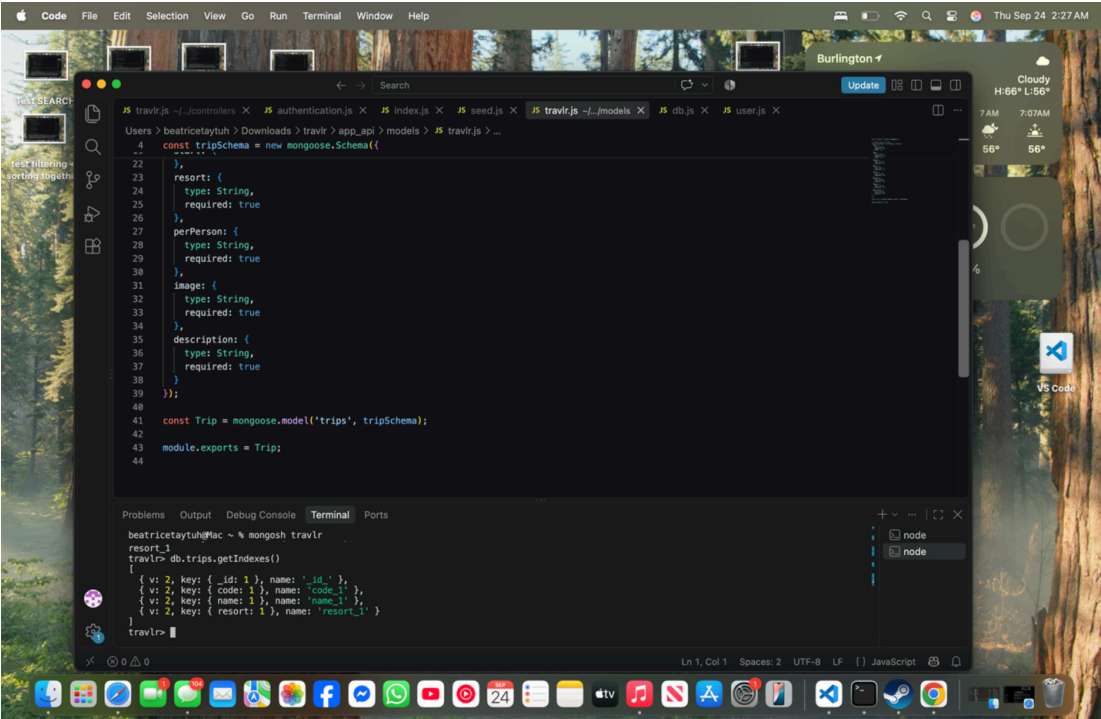
The enhanced application continues to retrieve trip records from MongoDB and display the stored trip information on the customer-facing Travel page.

Justification for Inclusion in the ePortfolio

I chose Travlr Getaways for my ePortfolio because it demonstrates how database design affects the reliability and usefulness of a full-stack application. Rather than treating MongoDB only as a place to store documents, this enhancement required me to consider the rules that should govern the data and the way the application will retrieve it. The enhanced schema uses required-field validation for the trip properties and strengthens the trip code by defining it as unique and indexed. I also added indexes to the trip code, name, and resort fields used for retrieval so the database can use indexed access paths instead of depending only on collection scanning as the amount of data grows.

I verified the database indexes directly in MongoDB. The index listing provides evidence that the collection contains indexes supporting important trip fields. This makes the database enhancement visible at the database level rather than relying only on the Mongoose source code as evidence.

In the final version of the enhancement, I also explicitly defined the resort index in the Mongoose trip schema using the `index: true` setting. This was important because verifying that an index exists in my current MongoDB collection does not by itself guarantee that the same index will be created when the application is configured with a new database. Defining the index in the schema makes the database design more reproducible and keeps the application model consistent with the indexes that I tested. This improvement also strengthened the connection between the application's source code and the database configuration.



The screenshot shows a VS Code editor with a Mongoose schema for a 'Trip' model. The schema includes fields for 'resort', 'perPerson', 'image', and 'description', each with a type of 'String' and 'required: true'. The 'resort' field is also indexed. The terminal window shows the command `travlr> db.trips.getIndexes()` and the output, which lists the indexes for the 'trips' collection: `{ v: 1, key: { _id: 1 }, name: '_id_' }`, `{ v: 2, key: { Code: 1 }, name: 'Code_1' }`, `{ v: 2, key: { name: 1 }, name: 'name_1' }`, and `{ v: 2, key: { resort: 1 }, name: 'resort_1' }`.

```
const tripSchema = new mongoose.Schema({
  resort: {
    type: String,
    required: true
  },
  perPerson: {
    type: String,
    required: true
  },
  image: {
    type: String,
    required: true
  },
  description: {
    type: String,
    required: true
  }
});

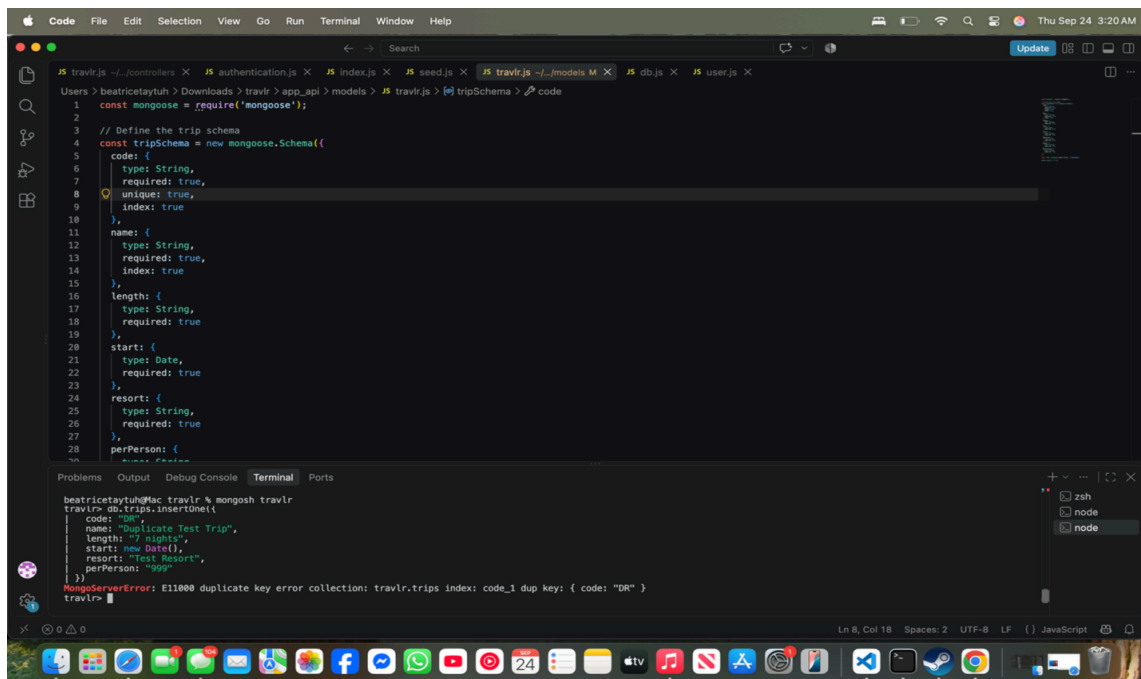
const Trip = mongoose.model('trips', tripSchema);
module.exports = Trip;
```

```
beatricetayuh@Mac ~ % mongosh travlr
resort_1
travlr> db.trips.getIndexes()
[
  { v: 1, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { Code: 1 }, name: 'Code_1' },
  { v: 2, key: { name: 1 }, name: 'name_1' },
  { v: 2, key: { resort: 1 }, name: 'resort_1' }
]
```

Figure 2. Database Index Verification.

MongoDB reports indexes for the trips collection, including indexes supporting trip code, trip name, and resort-related database access.

One of the most important improvements was strengthening the trip code. A trip code should identify a specific trip, so allowing two records to use the same code could create ambiguity and reduce data integrity. I defined the code field as required and unique and tested the rule by attempting to insert another record with the existing code DR. MongoDB rejected the operation with an E11000 duplicate-key error. This test demonstrates that the uniqueness requirement is enforced by the database and prevents the duplicate record from being stored.



```
1 const mongoose = require('mongoose');
2
3 // Define the trip schema
4 const tripSchema = new mongoose.Schema({
5   code: {
6     type: String,
7     required: true,
8     unique: true,
9     index: true
10  },
11  name: {
12    type: String,
13    required: true,
14    index: true
15  },
16  length: {
17    type: String,
18    required: true
19  },
20  start: {
21    type: Date,
22    required: true
23  },
24  resort: {
25    type: String,
26    required: true
27  },
28  perPerson: {
29    type: Number,
30    required: true
31  }
32 });
```

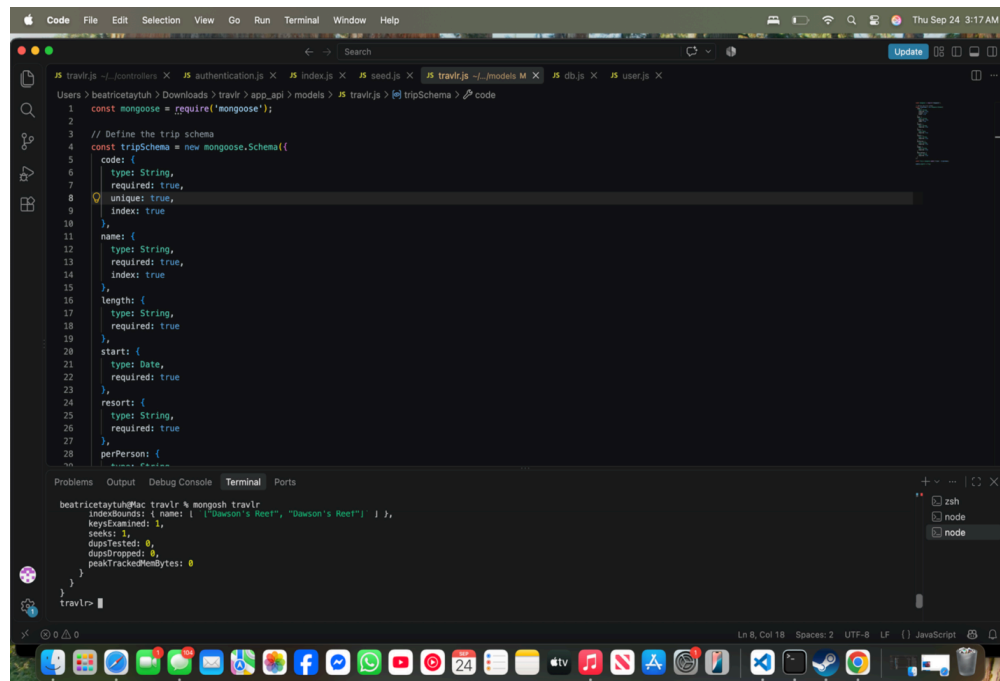
```
beatricetayuh@Mac travlr % mongosh travlr
travlr> db.trips.insertOne({
  code: "DR",
  name: "Duplicate Test Trip",
  length: "7 nights",
  start: new Date(),
  resort: "Test Resort",
  perPerson: "999"
})
MongoServerError: E11000 duplicate key error collection: travlr.trips index: code_1 dup key: { code: "DR" }
travlr>
```

Figure 3. Unique Trip Code Constraint Preventing Duplicate Records.

The enhanced Mongoose schema defines the trip code as required, unique, and indexed. MongoDB rejects an attempted duplicate DR record with an E11000 duplicate-key error.

I also evaluated the use of indexes with MongoDB execution plans. When I queried for Dawson's Reef, the execution information showed an IXSCAN using the name_1 index. The test

returned one record while examining the indexed key associated with the query. This provides concrete evidence that the name index can participate in the retrieval process instead of the application merely declaring an index in source code.



```
1 const mongoose = require('mongoose');
2
3 // Define the trip schema
4 const tripSchema = new mongoose.Schema({
5   codes: {
6     type: String,
7     required: true,
8     unique: true,
9     index: true
10  },
11   names: {
12     type: String,
13     required: true,
14     index: true
15   },
16   length: {
17     type: String,
18     required: true
19   },
20   start: {
21     type: Date,
22     required: true
23   },
24   resort: {
25     type: String,
26     required: true
27   },
28   perPerson: {
29     type: Number,
30     required: true
31   }
32 });
```

```
beatricetaytuh@Mac travlr % mongosh travlr
indexBounds: { name: { ["Dawson's Reef", "Dawson's Reef"] : 1 },
keyIsMinimized: 1,
seeks: 1,
dupTested: 0,
dropped: 0,
peakTrackedMemory: 0
}
travlr>
```

Figure 4. MongoDB Indexed Query Execution for Trip Name.

The execution plan for the Dawson's Reef query shows an IXSCAN using the name_1 index, demonstrating that MongoDB can use the added index when retrieving a matching trip name.

A similar verification was performed for resort-based retrieval. The execution information identifies the resort_1 index for a resort lookup. Together, the index tests demonstrate my ability to connect database design decisions with observable database behavior and to verify an enhancement through testing rather than assuming that a schema change is working as intended.

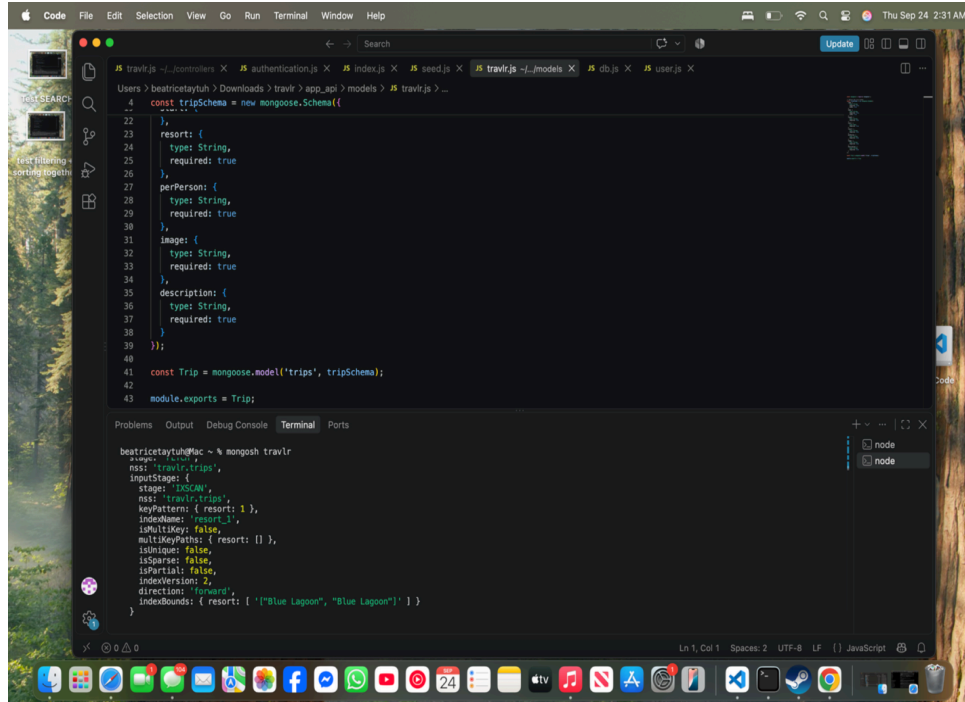


Figure 5. Verification of MongoDB Resort Index Usage.

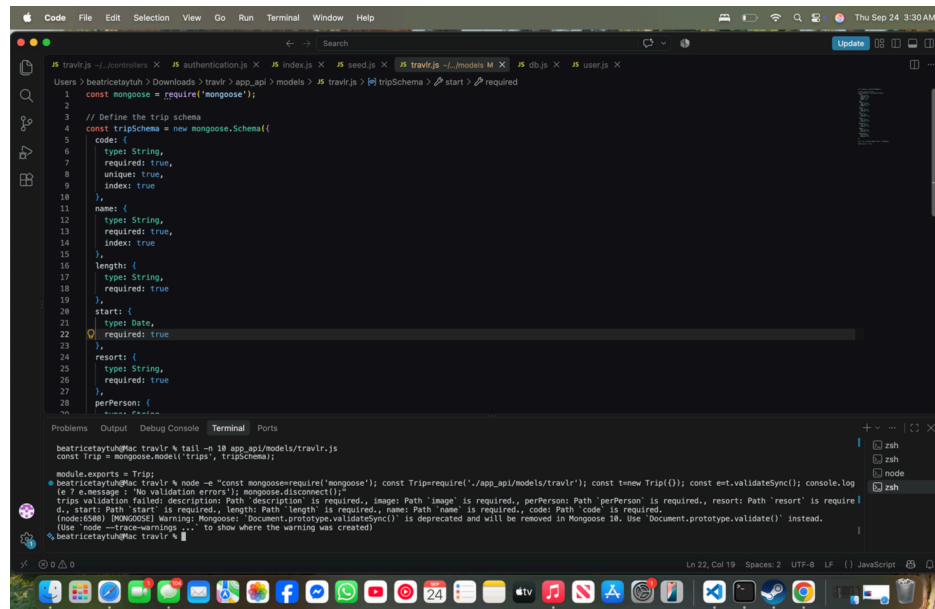
The query execution information identifies the resort_1 index for a resort-based lookup, providing additional evidence that the database enhancement supports indexed retrieval.

Course Outcomes

This enhancement supports the course outcome involving the use of well-founded techniques, skills, and tools to implement computing solutions that deliver value. MongoDB and Mongoose were used to strengthen the data layer through schema validation, uniqueness enforcement, and indexing. These changes improve the quality of the application's stored data and create a stronger foundation for retrieving trip information as the application grows.

The enhancement also supports the security-focused course outcome because database reliability depends on controlling what data is accepted. Required-field validation reduces the possibility that incomplete trip documents will be treated as valid records, while the unique trip-code rule

protects an important identifier from duplication. These controls do not replace a complete application-security strategy, but they demonstrate a security mindset by reducing avoidable data-integrity weaknesses at the database boundary.



```
const mongoose = require('mongoose');
// Define the trip schema
const tripSchema = new mongoose.Schema({
  code: {
    type: String,
    required: true,
    unique: true,
    index: true
  },
  name: {
    type: String,
    required: true,
    index: true
  },
  length: {
    type: String,
    required: true
  },
  start: {
    type: Date,
    required: true
  },
  resort: {
    type: String,
    required: true
  },
  perPerson: {
    type: Number,
    required: true
  }
});
```

```
beatrice@mac:~/travlr$ tail -n 10 app_api/models/travlr.js
const Trip = mongoose.model('trips', tripSchema);

module.exports = Trip;
beatrice@mac:~/travlr$ node -e "const mongoose=require('mongoose'); const Trip=require('./app_api/models/travlr'); const t=new Trip({}); const ent=validateSync(); console.log(
67 e.message); No validation errors"; mongoose.disconnect();
trips validation failed: description: Path `description` is required., image: Path `image` is required., perPerson: Path `perPerson` is required., resort: Path `resort` is require
(node:6586) [MongooseWarning] Warning: Mongoose `Document.prototype.validateSync()` is deprecated and will be removed in Mongoose 10. Use `Document.prototype.validate()` instead.
(Use `node --trace-warnings` to show where the warning was created)
```

Figure 6. Mongoose Required-Field Validation Test.

A test with missing trip values produces validation errors for required fields, demonstrating that incomplete trip records are rejected before they can be treated as valid application data.

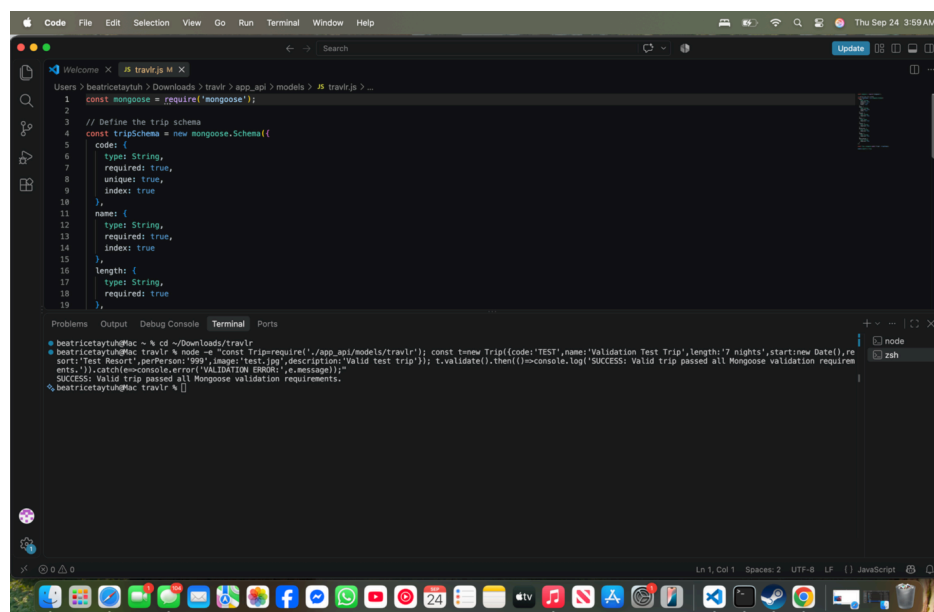
My original plan for this category was to improve validation, querying, indexing, data integrity, and integration between the application and database. The completed enhancement remains consistent with that plan. I strengthened validation and uniqueness requirements, added and verified indexes, tested database behavior, and confirmed that valid trip data continues to work with the application. The enhancement also complements the search, filtering, and sorting work completed in the earlier artifact enhancement because those application features depend on a reliable data layer.

Reflection on the Enhancement Process

The enhancement process helped me understand that a database improvement should be tested from more than one perspective. At first, adding required, unique, and index settings to the Mongoose schema appeared to be enough. Testing showed why verification is also necessary.

The duplicate-code test confirmed that MongoDB actually enforced the unique index, while the explain output allowed me to see whether MongoDB selected an index during a query. This made the enhancement more meaningful because I could connect the code I wrote with the behavior of the database.

Another important part of the process was testing both invalid and valid data. A useful validation rule must reject records that violate the schema without preventing properly formed records from being accepted. The required-field test produced validation errors when necessary values were missing, while the complete test record successfully passed validation. Testing both outcomes gave me greater confidence that the schema rules were functioning as intended.



```
1 const mongoose = require('mongoose');
2
3 // Define the trip schema
4 const tripSchema = new mongoose.Schema({
5   code: {
6     type: String,
7     required: true,
8     unique: true,
9     index: true
10  },
11  name: {
12    type: String,
13    required: true,
14    index: true
15  },
16  length: {
17    type: String,
18    required: true
19  },
20 });
```

```
beatrice@mac ~ % cd ~/Downloads/travlr
beatrice@mac travlr % node -e "const Trip=require('./app_api/models/travlr'); const t=new Trip({code:'TEST',name:'Validation Test Trip',length:'7 nights',start:new Date(),
sortsTest: {person:'person1',image:'test.jpg',description:'valid test trip'}}; t.validate().then(()=>console.log('SUCCESS: Valid trip passed all mongoose validation require
ments')).catch((e)=>console.error('VALIDATION ERROR:',e.message));"
SUCCESS: Valid trip passed all Mongoose validation requirements.
beatrice@mac travlr %
```

Figure 7. Successful Validation of Complete Trip Data.

A complete test record passes the Mongoose validation requirements, confirming that the validation rules reject incomplete data while still accepting properly formed trip records.

The main challenge was distinguishing between changes that existed in the application code and changes that could be proven at the database level. For example, seeing `index: true` in the schema shows my intent, but the MongoDB index listing and execution plans provide stronger evidence that the database contains and can use those indexes. Likewise, defining `unique: true` is more meaningful when an attempted duplicate insertion is actually rejected. Working through these tests strengthened my understanding of database validation, indexing, query execution, and data integrity.

An additional improvement I made during the final review of the enhancement was explicitly defining the resort index in the Mongoose schema. Although I had already verified that the `resort_1` index existed and could be used by MongoDB, I recognized that an index existing in my current database does not necessarily make the configuration reproducible in a new environment. Adding `index: true` to the resort field ensures that the schema more accurately represents the database design I intended and tested. This helped me better understand the importance of keeping application models and database configurations consistent rather than relying on settings that may exist only in a particular development environment.

Overall, this enhancement improved Travlr Getaways by making its data layer more deliberate and verifiable. It demonstrates my growth from simply connecting an application to MongoDB toward designing database behavior, enforcing data rules, and validating the results through targeted tests. These are skills that are relevant to my interest in data- and database-focused

computer science work because reliable applications depend on data that is stored consistently, protected from avoidable integrity problems, and retrievable through appropriate database structures.